

AI Study Buddy Backend Prompt Handbook

A sequenced library of copy-ready prompts for designing and building the complete application backend with Claude — from requirement analysis to file-by-file code generation.

TARGET STACK

Node.js

Express.js

MongoDB Atlas

Mongoose

JWT

bcryptjs

Multer

LangChain

Google Gemini API

ChromaDB

pdf-parse

Prepared June 2026 · 30 sequenced prompt sections · Architecture-first, code-last

About This Handbook

This handbook is a sequenced library of prompts for building the complete backend of the AI Study Buddy application using Claude. It contains no implementation code. Instead, each section gives you a precise, copy-ready prompt that directs Claude to design or document one part of the system, in the exact order a backend engineer should build the project.

Work through the sections top to bottom. Paste each Detailed Prompt into Claude, review the output against the Checklist, save the Deliverables, and carry the results forward as context for the next prompt. The final master prompt in Section 30 stitches every prior output together into a single, file-by-file code-generation request.

How to use it

- 1 Go in order.** The sections are dependency-ordered. Architecture decisions made early (folder structure, schemas, RAG flow) are referenced by later prompts, so skipping ahead will produce inconsistent output.
- 2 Feed outputs forward.** Each prompt assumes Claude can see the relevant results from earlier sections. Keep a running document of approved outputs and paste the pieces a prompt depends on alongside it.
- 3 Use the five-part frame.** Every section is structured as Objective, Expected Output, Detailed Prompt, Deliverables, and Checklist. Read the Objective first, run the Prompt, then verify with the Checklist before moving on.
- 4 Keep the stack fixed.** All prompts pin the same stack: Node.js, Express.js, MongoDB Atlas (Mongoose), JWT, bcryptjs, Multer, LangChain, Google Gemini API, ChromaDB, and pdf-parse. Restate it if you start a fresh Claude conversation so nothing drifts.
- 5 Treat the handbook as living.** When Claude proposes a better pattern, update your saved deliverables so downstream prompts inherit the improvement.

Contents

01	Backend Requirement Analysis
02	System Architecture
03	Backend Folder Structure
04	Database Design
05	Authentication System
06	Authentication API
07	PDF Upload Architecture
08	PDF Processing
09	Chunking Strategy
10	Embedding Generation
11	ChromaDB Architecture
12	RAG Pipeline
13	Gemini Integration
14	Chat System Architecture
15	Chat Database Design
16	Chat APIs
17	Quiz Generation Architecture
18	Quiz APIs
19	Flashcard Architecture
20	Flashcard APIs
21	Dashboard Analytics
22	Dashboard API
23	Middleware Architecture
24	Error Handling
25	Validation Architecture
26	Security Architecture
27	Environment Configuration
28	Testing Strategy
29	Backend Development Roadmap
30	Final Backend Code Generation

Backend Requirement Analysis

OBJECTIVE

Establish a shared, unambiguous understanding of what the backend must do before any architecture or code is designed.

EXPECTED OUTPUT

A structured requirements document covering scope, functional and non-functional requirements, a feature breakdown, user workflows, and a clear statement of backend responsibilities.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Principal Backend Architect and Business Analyst. I am building the backend for an AI Study Buddy application using Node.js, Express.js, MongoDB Atlas (Mongoose), JWT, bcryptjs, Multer, LangChain, Google Gemini API, ChromaDB, and pdf-parse. Students upload study PDFs and then chat with them, generate quizzes and flashcards, and track progress on a dashboard.

Produce a complete Backend Requirement Analysis with these labelled sections: (1) Backend Scope – what the backend owns and explicitly excludes; (2) Functional Requirements – numbered, testable statements; (3) Non-Functional Requirements – performance, security, scalability, availability, cost; (4) Feature Breakdown – every feature decomposed into backend capabilities; (5) User Workflows – step-by-step journeys for upload, chat, quiz, flashcard, and dashboard; (6) Backend Responsibilities – the concrete duties of each subsystem. Use tables where they add clarity and keep every requirement atomic and verifiable.
```

DELIVERABLES

- ▶ Requirement analysis document (scope, FRs, NFRs)
- ▶ Feature breakdown table
- ▶ User workflow descriptions for all five core features

CHECKLIST

- ☐ Scope clearly lists inclusions and exclusions
- ☐ Every functional requirement is numbered and testable
- ☐ Non-functional requirements include security and performance targets
- ☐ All five core features are decomposed
- ☐ User workflows cover the full upload-to-dashboard journey

OBJECTIVE

Define the high-level shape of the system and how every service, request, and data flow connects — including the RAG, Gemini, and ChromaDB paths.

EXPECTED OUTPUT

A narrated architecture document with diagrams describing components, their interactions, and the major request and data flows.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Principal Backend Architect. Using the approved requirement analysis and the stack Node.js, Express.js, MongoDB Atlas (Mongoose), JWT, bcryptjs, Multer, LangChain, Google Gemini API, ChromaDB, and pdf-parse, design the system architecture for the AI Study Buddy backend.

Deliver: (1) a High-Level Architecture overview of all components (API layer, auth, file handling, processing pipeline, vector store, LLM service, database); (2) a Service Interaction Diagram; (3) the Request Flow for a typical authenticated call; (4) the Data Flow from PDF upload to stored embeddings; (5) the RAG Flow from user question to grounded answer; (6) the Gemini Integration Flow; and (7) the ChromaDB Flow. Render each diagram as a clear ASCII / Mermaid diagram with a short narrative explaining the boundaries, responsibilities, and failure points of each component.
```

DELIVERABLES

- ▶ High-level architecture diagram
- ▶ Service interaction diagram
- ▶ Request, data, RAG, Gemini, and ChromaDB flow diagrams
- ▶ Narrative explaining each component boundary

CHECKLIST

- ☐ All listed components appear in the architecture
- ☐ Diagrams are included and readable
- ☐ RAG, Gemini, and ChromaDB flows are each shown end to end
- ☐ Component responsibilities and boundaries are described
- ☐ Failure points and trust boundaries are identified

Backend Folder Structure

OBJECTIVE

Lock in a scalable, convention-driven project layout so every later file has an obvious home.

EXPECTED OUTPUT

A complete annotated folder tree using MVC plus a service layer, with the responsibility of every directory explained.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Node.js Engineer. Design a scalable backend folder structure for the AI Study Buddy app built on Node.js, Express.js, MongoDB Atlas (Mongoose), JWT, bcryptjs, Multer, LangChain, Google Gemini API, ChromaDB, and pdf-parse.

Produce: (1) a full directory tree shown as ASCII; (2) an MVC layout (routes, controllers, models); (3) a dedicated Service Layer for business logic (auth, pdf, embeddings, rag, gemini, chat, quiz, flashcard, dashboard); (4) a Utility Layer; (5) a Configuration Layer; and (6) a Middleware Layer. For every folder, write one or two sentences on its single responsibility and what does and does not belong there. Favour separation of concerns and predictable naming so the structure scales to dozens of files without ambiguity.
```

DELIVERABLES

- ▶ Annotated folder tree
- ▶ Responsibility note for every directory
- ▶ Defined locations for routes, controllers, services, utils, config, and middleware

CHECKLIST

- ☐ Tree uses MVC plus a service layer
- ☐ Each AI subsystem has a dedicated service file location
- ☐ Config, utils, and middleware layers are present
- ☐ Every folder has a stated single responsibility
- ☐ Naming conventions are consistent and predictable

OBJECTIVE

Design correct, validated, well-indexed MongoDB schemas for the five core domain entities.

EXPECTED OUTPUT

Mongoose schema designs for User, Note, Chat, Quiz, and Flashcard with fields, validation, relationships, and indexing strategy for each.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Database Architect. Design MongoDB schemas (Mongoose) for the AI Study Buddy backend. Create schemas for: User, Note, Chat, Quiz, and Flashcard.

For each schema provide: (1) Fields – name, type, required flag, and default; (2) Validation Rules – formats, lengths, enums, and custom validators; (3) Relationships – references between collections and the ownership model (every document belongs to a user); and (4) Indexing Strategy – which fields are indexed, which are compound, and why, framed around the app's real query patterns. Present each schema as a clear field table followed by the indexing rationale. Do not write application logic – only the schema design.
```

DELIVERABLES

- ▶ Schema tables for User, Note, Chat, Quiz, Flashcard
- ▶ Validation rules per field
- ▶ Relationship and ownership map
- ▶ Indexing strategy with rationale

CHECKLIST

- ☐ All five schemas are defined with typed fields
- ☐ Validation rules are specified per field
- ☐ Relationships and per-user ownership are explicit
- ☐ Indexes are justified by query patterns
- ☐ Compound indexes are identified where useful

Authentication System

OBJECTIVE

Design a secure, complete authentication and authorization model before exposing any auth endpoints.

EXPECTED OUTPUT

An end-to-end auth design covering registration, login, JWT issuance, password hashing, protected routes, and authorization, with example request/response payloads.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

Act as a Senior Backend Security Engineer. Design the authentication system for the AI Study Buddy backend using JWT and bcryptjs on Express.

Cover: (1) User Registration; (2) User Login; (3) JWT Authentication (token contents, signing, expiry, refresh strategy); (4) Password Hashing with bcryptjs (salt rounds, where hashing happens); (5) Protected Routes (how the verify middleware attaches the user); and (6) Authorization Flow (ensuring users only access their own resources). Include example request and response JSON for register and login, and describe the token lifecycle. Focus on design and security reasoning, not implementation code.

DELIVERABLES

- ▶ Auth design document
- ▶ Token lifecycle description
- ▶ Example request/response payloads for register and login
- ▶ Authorization (ownership) rules

CHECKLIST

- ☐ Registration and login flows are fully described
- ☐ JWT contents, expiry, and refresh approach are defined
- ☐ Password hashing strategy uses bcryptjs with stated salt rounds
- ☐ Protected-route mechanism is explained
- ☐ Per-user authorization is enforced in the design

Authentication API

OBJECTIVE

Specify the exact contract for each auth endpoint so the frontend and tests can rely on it.

EXPECTED OUTPUT

A precise API specification for register, login, me, and logout, with validation rules, error handling, and success responses.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

Act as a Senior API Designer. Using the approved authentication design, specify these Express endpoints for the AI Study Buddy backend: POST /register, POST /login, GET /me, POST /logout.

For each endpoint document: the request body and headers; Validation rules for every input; Error Handling with specific status codes and error bodies (e.g. 400 validation, 401 unauthenticated, 409 duplicate email); and the Success Response shape with an example. Keep response formats consistent across all four endpoints and note which require a valid JWT. Provide the specification as structured tables and JSON examples, not implementation code.

DELIVERABLES

- ▶ Endpoint specs for /register, /login, /me, /logout
- ▶ Validation table per endpoint
- ▶ Error catalogue with status codes
- ▶ Success response examples

CHECKLIST

- ☐ All four endpoints are specified
- ☐ Validation rules are listed for each input
- ☐ Error responses use consistent status codes and shapes
- ☐ Success responses include example JSON
- ☐ Auth requirement per endpoint is stated

PDF Upload Architecture

OBJECTIVE

Design how study PDFs enter the system safely and how their files and metadata are stored.

EXPECTED OUTPUT

A complete upload workflow covering the upload flow, storage strategy, Multer configuration approach, and metadata persistence.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Backend Engineer. Design the PDF upload architecture for the AI Study Buddy backend using Multer on Express.

Design: (1) the PDF Upload Flow from client request to a stored, registered file; (2) a File Storage Strategy (local disk vs cloud, naming, per-user isolation, size and type limits); (3) the Multer Configuration approach (storage engine, file filter accepting only PDFs, limits); and (4) Metadata Storage (what is recorded in MongoDB – owner, original name, stored path, size, status, timestamps). Present the full workflow as a numbered sequence plus a diagram, and call out validation and security checks at each step. Design only – no code.
```

DELIVERABLES

- ▶ Upload workflow diagram and sequence
- ▶ Storage strategy decision
- ▶ Multer configuration plan (filter + limits)
- ▶ Metadata schema for uploaded files

CHECKLIST

- ☐ Upload flow is shown step by step
- ☐ Storage strategy isolates files per user
- ☐ Only PDFs are accepted and size limits are set
- ☐ Metadata fields are defined for MongoDB
- ☐ Security checks are placed at each step

PDF Processing

OBJECTIVE

Turn an uploaded PDF into clean, usable text ready for chunking.

EXPECTED OUTPUT

A processing pipeline design for parsing, extraction, cleaning, and error handling, using pdf-parse.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Backend Engineer. Design the PDF processing pipeline for the AI Study Buddy backend using pdf-parse.

Design: (1) PDF Parsing (reading the stored file and extracting raw text per page); (2) Text Extraction (assembling page text with page markers for later citation); (3) Text Cleaning (normalising whitespace, fixing hyphenation and broken lines, stripping headers/footers and artefacts, handling unicode); and (4) Error Handling (encrypted, image-only, corrupt, or empty PDFs, with status updates back to the file record). Present the pipeline as an ordered set of stages with inputs, outputs, and failure behaviour for each. Design only – no code.
```

DELIVERABLES

- ▶ Processing pipeline stages
- ▶ Text-cleaning rules
- ▶ Page-aware extraction approach
- ▶ Error-handling matrix for bad PDFs

CHECKLIST

- ☐ Parsing and extraction stages are defined
- ☐ Page markers are preserved for citation
- ☐ Cleaning rules handle whitespace, hyphenation, and artefacts
- ☐ Image-only and corrupt PDFs are handled gracefully
- ☐ File status is updated on success and failure

Chunking Strategy

OBJECTIVE

Decide how cleaned text is split into retrievable chunks that balance context and precision.

EXPECTED OUTPUT

A justified chunking design specifying size, overlap, storage, and metadata, with the reasoning behind each choice.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior GenAI Engineer. Design the text chunking strategy for the AI Study Buddy RAG pipeline using LangChain text splitters.

Design: (1) Chunk Size (recommended token/character size and why it fits Gemini's context and retrieval quality); (2) Chunk Overlap (amount and why, to preserve continuity); (3) Chunk Storage (how chunks map to ChromaDB documents); and (4) Chunk Metadata (user id, note/document id, page number, chunk index, source filename). Explain the trade-offs of larger vs smaller chunks for this study-notes use case and recommend a concrete default with the reasoning behind it. Design only – no code.
```

DELIVERABLES

- ▶ Recommended chunk size and overlap
- ▶ Chunk-to-vector storage mapping
- ▶ Chunk metadata schema
- ▶ Written rationale for the strategy

CHECKLIST

- ☐ Chunk size is recommended with justification
- ☐ Overlap value is specified and explained
- ☐ Metadata captures user, document, page, and index
- ☐ Trade-offs of chunk sizing are discussed
- ☐ Strategy fits Gemini context limits

Embedding Generation

OBJECTIVE

Define how chunks become vector embeddings, including model choice and performance handling.

EXPECTED OUTPUT

An embedding workflow with model selection, metadata association, and performance considerations.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior GenAI Engineer. Design the embedding generation workflow for the AI Study Buddy backend using LangChain with Google Gemini embeddings.

Design: (1) the Embedding Workflow (from cleaned chunks to stored vectors); (2) Embedding Model Selection (recommend a Gemini embedding model, with dimensionality and why it suits study content); (3) Metadata Association (attaching chunk metadata to each vector so retrieval can filter and cite); and (4) Performance Considerations (batching, rate limits, retries, idempotency, and avoiding re-embedding unchanged content). Present the workflow as an ordered pipeline and note where failures are retried or surfaced. Design only – no code.
```

DELIVERABLES

- ▶ Embedding pipeline definition
- ▶ Recommended embedding model and dimensions
- ▶ Vector metadata mapping
- ▶ Performance and batching plan

CHECKLIST

- ☐ End-to-end embedding workflow is defined
- ☐ A specific Gemini embedding model is recommended
- ☐ Metadata is attached to every vector
- ☐ Batching, rate limits, and retries are addressed
- ☐ Re-embedding of unchanged content is avoided

ChromaDB Architecture

OBJECTIVE

Design how vectors are organised, stored, searched, and filtered in ChromaDB.

EXPECTED OUTPUT

A ChromaDB architecture covering setup, collections, vector storage, similarity search, and metadata filtering, with diagrams.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Vector Database Engineer. Design the ChromaDB architecture for the AI Study Buddy backend.

Design: (1) ChromaDB Setup (deployment mode and how the Node backend connects); (2) Collections (collection-per-user vs shared collection with user metadata – recommend one and justify it); (3) Vector Storage (document, embedding, and metadata layout); (4) Similarity Search (distance metric, top-k, and score thresholds); and (5) Metadata Filtering (restricting retrieval to a user and optionally a specific document). Include architecture diagrams of the storage and query paths. Recommend concrete defaults and explain the isolation and scaling implications. Design only – no code.
```

DELIVERABLES

- ▶ ChromaDB setup and connection plan
- ▶ Collection design decision
- ▶ Vector + metadata storage layout
- ▶ Similarity search and filtering configuration
- ▶ Storage and query diagrams

CHECKLIST

- ☐ Setup and connection approach are defined
- ☐ Collection strategy is chosen and justified
- ☐ User isolation is enforced via metadata or collections
- ☐ Distance metric and top-k are specified
- ☐ Diagrams show storage and query paths

RAG Pipeline

OBJECTIVE

Assemble retrieval, context building, and generation into one coherent answer pipeline.

EXPECTED OUTPUT

An end-to-end RAG design: retrieval, query flow, context retrieval, prompt construction, and response generation.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior GenAI Engineer. Design the end-to-end RAG pipeline for the AI Study Buddy backend using LangChain, ChromaDB, and Google Gemini.

Design: (1) the Retrieval Pipeline (embed the query, search ChromaDB with user/document filters, return top-k chunks with scores); (2) the Query Flow (how a chat question travels through the system); (3) Context Retrieval (selecting, de-duplicating, and ordering chunks within a token budget); (4) Prompt Construction (system instructions, retrieved context, chat history, and the user question, plus a citation format); and (5) Response Generation (calling Gemini and returning a grounded answer with sources). Present the complete workflow as a single annotated diagram plus a stage-by-stage description, including behaviour when no relevant context is found. Design only – no code.
```

DELIVERABLES

- ▶ End-to-end RAG diagram
- ▶ Retrieval and filtering logic
- ▶ Context assembly and token-budgeting rules
- ▶ Prompt template structure with citations
- ▶ No-context fallback behaviour

CHECKLIST

- ☐ Query is embedded and filtered to the user's data
- ☐ Top-k retrieval with scoring is defined
- ☐ Context is de-duplicated and fits a token budget
- ☐ Prompt template includes context, history, and question
- ☐ Grounded answer returns citations and handles empty results

Gemini Integration

OBJECTIVE

Design a clean, resilient service layer around the Google Gemini API.

EXPECTED OUTPUT

A Gemini service-layer design covering API interaction, prompt strategy, error handling, and rate-limit handling.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

Act as a Senior GenAI Engineer. Design the Google Gemini integration service layer for the AI Study Buddy backend.

Design: (1) the Gemini Service Layer (a single module that wraps all Gemini calls so the rest of the app never touches the SDK directly); (2) API Interaction (chat completions and embeddings, model and parameter choices such as temperature and max tokens); (3) a Prompt Engineering Strategy (reusable system prompts for chat, quiz, and flashcards, with guardrails to stay grounded in retrieved context); (4) Error Handling (timeouts, safety blocks, malformed output, and fallbacks); and (5) Rate Limit Handling (exponential backoff, retries, concurrency limits, and quota awareness). Present this as an interface definition and a set of policies. Design only – no code.

DELIVERABLES

- ▶ Gemini service interface definition
- ▶ Model and parameter choices
- ▶ Reusable prompt templates
- ▶ Error and rate-limit policies

CHECKLIST

- ☐ All Gemini access is centralised in one service
- ☐ Chat and embedding interactions are both covered
- ☐ Reusable system prompts exist for each feature
- ☐ Errors and safety blocks are handled with fallbacks
- ☐ Backoff and rate-limit handling are defined

Chat System Architecture

OBJECTIVE

Design the conversational layer: starting, continuing, listing, renaming, and deleting chats.

EXPECTED OUTPUT

A chat system design with flow diagrams for new chat, continue chat, history, delete, and rename.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

Act as a Senior Backend Architect. Design the chat system architecture for the AI Study Buddy backend, where each chat is grounded in the user's uploaded documents via the RAG pipeline.

Design these flows, each with a diagram: (1) New Chat (create a conversation, optionally scoped to one or more documents); (2) Continue Chat (append a message, run retrieval, generate a grounded reply, persist both turns); (3) Chat History (list a user's conversations and load one conversation's messages); (4) Delete Chat (remove a conversation and its messages with ownership checks); and (5) Rename Chat (update a conversation title). Show how chat history feeds back into prompt construction and how ownership is enforced on every operation. Design only – no code.

DELIVERABLES

- ▶ Flow diagrams for all five chat operations
- ▶ How history feeds the RAG prompt
- ▶ Ownership enforcement model
- ▶ Document-scoping approach for chats

CHECKLIST

- ☐ All five operations have flow diagrams
- ☐ Continue-chat integrates retrieval and persistence
- ☐ History is scoped and loaded per conversation
- ☐ Ownership is checked on every operation
- ☐ Chat history is fed back into prompt construction

Chat Database Design

OBJECTIVE

Model conversations and messages efficiently for fast history loading and clean relationships.

EXPECTED OUTPUT

Mongoose designs for the Chat and Message schemas with relationships and query strategy.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

Act as a Senior Database Architect. Design the chat persistence model for the AI Study Buddy backend in MongoDB (Mongoose).

Design: (1) the Chat Schema (owner, title, document scope, timestamps, last-activity); (2) the Message Schema (chat reference, role of user/assistant, content, retrieved-source references for assistant turns, timestamp); (3) Relationships (how messages relate to chats and users, and whether to embed or reference messages – recommend one with reasoning); and (4) Query Strategy (indexes and access patterns for listing chats, loading a conversation in order, and pagination). Present schemas as field tables with the indexing and embed-vs-reference rationale. Design only – no code.

DELIVERABLES

- ▶ Chat and Message schema tables
- ▶ Embed-vs-reference decision with rationale
- ▶ Relationship map
- ▶ Indexes and pagination strategy

CHECKLIST

- ☐ Chat and Message schemas are fully specified
- ☐ Embed-vs-reference choice is justified
- ☐ Assistant messages can store source references
- ☐ Indexes support listing and ordered loading
- ☐ Pagination strategy is defined

Chat APIs

OBJECTIVE

Specify the chat endpoint contracts the frontend will consume.

EXPECTED OUTPUT

API specifications for ask, list, get, rename, and delete chat endpoints, with request/response examples.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior API Designer. Specify these chat endpoints for the AI Study Buddy backend (all require a valid JWT and enforce ownership):
POST /chat/ask, GET /chats, GET /chat/:id, PATCH /chat/:id, DELETE /chat/:id.

For each endpoint document the request (params, body, headers), validation, error handling with status codes, and a success response with
example JSON. For POST /chat/ask, show the request carrying the question and optional chatId/document scope, and a response containing the
grounded answer plus source citations. Keep response envelopes consistent across all endpoints. Provide tables and JSON examples – no
implementation code.
```

DELIVERABLES

- ▶ Specs for all five chat endpoints
- ▶ Request/response JSON examples
- ▶ Validation and error catalogue
- ▶ Citation shape in the ask response

CHECKLIST

- ☐ All five endpoints are specified
- ☐ ask returns a grounded answer with sources
- ☐ Ownership and JWT requirements are stated
- ☐ Validation and error responses are consistent
- ☐ Example JSON is provided for each endpoint

Quiz Generation Architecture

OBJECTIVE

Design reliable generation of validated multiple-choice quizzes from a user's documents.

EXPECTED OUTPUT

A quiz design covering the generation flow, MCQ structure, validation, and storage.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior GenAI Engineer. Design the quiz generation architecture for the AI Study Buddy backend using retrieved document context and Google Gemini.

Design: (1) the Quiz Generation Flow (select a document or topic, retrieve representative context, prompt Gemini for N questions at a chosen difficulty); (2) the MCQ Structure (question, options, correct answer key, explanation, source reference) with a strict JSON output schema; (3) Question Validation (enforce one correct option, unique non-empty distractors, parseable JSON, and reject malformed items with a retry); and (4) Quiz Storage (how generated quizzes and their questions are persisted for later attempts). Present the flow, the JSON schema, and the validation rules. Design only – no code.
```

DELIVERABLES

- ▶ Quiz generation flow
- ▶ Strict MCQ JSON schema
- ▶ Validation rules and retry policy
- ▶ Quiz storage model

CHECKLIST

- ☐ Generation flow uses retrieved context
- ☐ MCQ schema is strict and parseable
- ☐ Validation enforces exactly one correct answer
- ☐ Malformed output triggers retry or rejection
- ☐ Quizzes are persisted for later attempts

Quiz APIs

OBJECTIVE

Specify endpoints for generating quizzes, submitting answers, and reviewing history.

EXPECTED OUTPUT

API specs for generate, submit, and history quiz endpoints with request/response examples.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior API Designer. Specify these quiz endpoints for the AI Study Buddy backend (JWT-protected, ownership-enforced):  
POST /quiz/generate, POST /quiz/submit, GET /quiz/history.  
  
For each endpoint document request, validation, error handling, and success response with example JSON. For generate, accept a document/topic, question count, and difficulty, and return the quiz (without leaking answers if used for live attempts). For submit, accept the quizId and the user's answers and return a score, per-question correctness, and explanations. For history, return past attempts with scores and dates. Keep response envelopes consistent. Tables and JSON only – no code.
```

DELIVERABLES

- ▶ Specs for generate, submit, history
- ▶ Scoring response shape
- ▶ Validation and error catalogue
- ▶ Answer-leak handling for live quizzes

CHECKLIST

- ☐ All three endpoints are specified
- ☐ generate accepts count and difficulty
- ☐ submit returns score and per-question results
- ☐ history returns past attempts with dates
- ☐ Validation and errors are consistent

Flashcard Architecture

OBJECTIVE

Design generation, storage, and retrieval of study flashcards.

EXPECTED OUTPUT

A flashcard design covering generation, storage, and retrieval strategy.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior GenAI Engineer. Design the flashcard architecture for the AI Study Buddy backend using retrieved document context and Google Gemini.

Design: (1) Flashcard Generation (retrieve key context from a document, prompt Gemini for concise front/back cards with a strict JSON schema, and validate them); (2) Flashcard Storage (owner, source document, front, back, optional tags/difficulty, timestamps); and (3) a Retrieval Strategy (listing a user's cards, filtering by document or tag, ordering, and pagination). Recommend the card JSON schema and the persistence model, and note de-duplication of near-identical cards. Design only – no code.
```

DELIVERABLES

- ▶ Flashcard generation flow and JSON schema
- ▶ Flashcard storage model
- ▶ Retrieval, filtering, and pagination strategy

CHECKLIST

- ☐ Generation uses retrieved context and a strict schema
- ☐ Cards are validated before storage
- ☐ Storage model captures source and ownership
- ☐ Retrieval supports filtering and pagination
- ☐ Near-duplicate cards are de-duplicated

Flashcard APIs

OBJECTIVE

Specify endpoints to generate, list, and delete flashcards.

EXPECTED OUTPUT

API specs for generate, list, and delete flashcard endpoints with request/response examples.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior API Designer. Specify these flashcard endpoints for the AI Study Buddy backend (JWT-protected, ownership-enforced):  
POST /flashcards/generate, GET /flashcards, DELETE /flashcards/:id.  
  
For each endpoint document request, validation, error handling, and success response with example JSON. For generate, accept a source  
document and an optional count and return the created cards. For list, support filtering by document or tag plus pagination. For delete,  
enforce ownership and return a clear success or 404. Keep response envelopes consistent across endpoints. Tables and JSON only – no code.
```

DELIVERABLES

- ▶ Specs for generate, list, delete
- ▶ Filtering and pagination on list
- ▶ Validation and error catalogue
- ▶ Ownership handling on delete

CHECKLIST

- ☐ All three endpoints are specified
- ☐ generate returns created cards
- ☐ list supports filtering and pagination
- ☐ delete enforces ownership and handles 404
- ☐ Response envelopes are consistent

Dashboard Analytics

OBJECTIVE

Define the metrics that summarise a student's activity and progress.

EXPECTED OUTPUT

A definition of every dashboard metric and how each is computed from the data model.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Backend Engineer and Data Analyst. Design the dashboard analytics for the AI Study Buddy backend, computed per authenticated user.

Define each metric and its exact computation: (1) PDFs Uploaded Analytics (total uploads, processing status breakdown, recent uploads); (2) Quiz Statistics (quizzes taken, questions answered, accuracy); (3) Average Score Calculation (how scores are aggregated across attempts, and the time window); (4) Flashcard Statistics (total cards, cards by document); and (5) Chat Statistics (conversation and message counts, activity over time). For each metric, state the source collections, the grouping, and any edge cases (e.g. no data yet). Design only – no code.
```

DELIVERABLES

- ▶ Metric catalogue with formulas
- ▶ Source collection per metric
- ▶ Time-window definitions
- ▶ Empty-state handling

CHECKLIST

- ☐ All five metric groups are defined
- ☐ Each metric states its source and formula
- ☐ Average score aggregation is precise
- ☐ Time windows are specified where relevant
- ☐ Empty-state behaviour is defined

Dashboard API

OBJECTIVE

Specify a single efficient dashboard endpoint backed by aggregation pipelines.

EXPECTED OUTPUT

A specification for GET /dashboard with aggregation pipeline design, performance optimisation, and a response structure.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

Act as a Senior API Designer and MongoDB Specialist. Specify the GET /dashboard endpoint for the AI Study Buddy backend (JWT-protected, scoped to the current user).

Provide: (1) Aggregation Pipelines – outline the MongoDB aggregation stages used to compute the metrics from Section 21, preferring a small number of efficient pipelines over many round trips; (2) Performance Optimisation – indexes the pipelines rely on, use of \$facet to compute several metrics in one pass, projection, and optional caching; and (3) Response Structure – a single consolidated JSON object grouping all metrics, with an example payload and an empty-state example. Present pipeline stages descriptively and give example JSON – no implementation code.

DELIVERABLES

- ▶ GET /dashboard specification
- ▶ Aggregation pipeline outline (\$facet)
- ▶ Index and caching plan
- ▶ Consolidated response JSON example

CHECKLIST

- ☐ Endpoint is scoped to the current user
- ☐ Pipelines compute all dashboard metrics
- ☐ \$facet or equivalent minimises round trips
- ☐ Required indexes are listed
- ☐ Response JSON and empty-state are shown

Middleware Architecture

OBJECTIVE

Design the cross-cutting middleware that every request passes through.

EXPECTED OUTPUT

Designs for authentication, error, validation, and logging middleware and their ordering.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

Act as a Senior Node.js Engineer. Design the Express middleware architecture for the AI Study Buddy backend.

Design: (1) Authentication Middleware (verify the JWT, load the user, attach it to the request, reject invalid tokens); (2) Error Middleware (a central handler that converts thrown errors into the standard API error format); (3) Validation Middleware (a reusable wrapper that validates request bodies, params, and queries against a schema); and (4) Logging Middleware (structured request/response logging with a correlation id, excluding secrets). Specify the responsibility, inputs, outputs, and the correct registration order of each middleware in the Express pipeline. Design only – no code.

DELIVERABLES

- ▶ Four middleware designs
- ▶ Responsibility and I/O for each
- ▶ Correct middleware ordering
- ▶ Correlation-id logging approach

CHECKLIST

- ☐ Auth middleware verifies JWT and loads the user
- ☐ Central error middleware uses the standard format
- ☐ Validation middleware is reusable across routes
- ☐ Logging is structured and omits secrets
- ☐ Registration order is specified

Error Handling

OBJECTIVE

Standardise how the backend represents and handles every error.

EXPECTED OUTPUT

A global error strategy, a consistent API error format, and an exception handling approach.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Backend Architect. Design the error handling strategy for the AI Study Buddy backend.

Design: (1) a Global Error Strategy (a custom error class hierarchy, operational vs programmer errors, and how errors propagate to the central handler); (2) an API Error Format (a single consistent JSON envelope with code, message, details, and request id, plus a status-code mapping table for common cases such as validation, auth, not-found, conflict, rate-limit, and upstream LLM failures); and (3) Exception Handling (async error capture, handling of unhandled rejections and uncaught exceptions, and safe messages that never leak internals). Present the error class hierarchy and the format with examples. Design only – no code.
```

DELIVERABLES

- ▶ Error class hierarchy
- ▶ Standard error JSON envelope
- ▶ Status-code mapping table
- ▶ Async and process-level exception handling

CHECKLIST

- ☐ Operational vs programmer errors are distinguished
- ☐ A single error envelope is defined
- ☐ Status-code mapping covers common cases
- ☐ Async errors reach the central handler
- ☐ Error messages never leak internals

Validation Architecture

OBJECTIVE

Design layered validation and sanitisation for every inbound request.

EXPECTED OUTPUT

A validation design covering request validation, input sanitisation, and security validation.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Backend Security Engineer. Design the request validation architecture for the AI Study Buddy backend.

Design: (1) Request Validation (a schema-driven approach – recommend a library such as Joi or Zod – validating body, params, and query with reusable schemas per route, returning structured field errors); (2) Input Sanitisation (trimming, normalising, stripping dangerous characters, and guarding against NoSQL injection and oversized payloads); and (3) Security Validation (file-type and size checks for uploads, enforcing ownership ids from the token rather than the body, and rejecting unexpected fields). Map which validations run in middleware vs the service layer. Design only – no code.
```

DELIVERABLES

- ▶ Schema-driven validation plan
- ▶ Sanitisation rules
- ▶ Security validation checklist
- ▶ Middleware-vs-service placement map

CHECKLIST

- ☐ A validation library is recommended
- ☐ Body, params, and query are all validated
- ☐ NoSQL injection and oversized payloads are guarded
- ☐ Ownership comes from the token, not the body
- ☐ Unexpected fields are rejected

Security Architecture

OBJECTIVE

Consolidate the backend's defensive posture across auth, transport, and abuse prevention.

EXPECTED OUTPUT

A security design covering JWT, password, rate limiting, CORS, and general API protection.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Application Security Engineer. Design the security architecture for the AI Study Buddy backend.

Design: (1) JWT Security (signing algorithm, secret management, short expiries, refresh handling, and revocation strategy); (2) Password Security (bcryptjs salt rounds, strength policy, and safe reset flow); (3) Rate Limiting (global and per-endpoint limits, with stricter limits on auth and LLM endpoints to control cost and abuse); (4) CORS (allowed origins, methods, and credentials policy); and (5) API Protection (security headers via helmet, payload size limits, dependency hygiene, and protection of the Gemini and Mongo credentials). Present each as a concrete policy with recommended defaults. Design only – no code.
```

DELIVERABLES

- ▶ JWT and password security policies
- ▶ Rate-limit configuration
- ▶ CORS policy
- ▶ API hardening checklist

CHECKLIST

- ☐ JWT signing, expiry, and revocation are specified
- ☐ Password policy and bcrypt rounds are set
- ☐ Rate limits protect auth and LLM endpoints
- ☐ CORS policy is explicit
- ☐ Security headers and secret handling are covered

Environment Configuration

OBJECTIVE

Centralise configuration and secrets so the app runs cleanly across environments.

EXPECTED OUTPUT

A configuration design covering environment variables, config structure, and secrets management.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Backend Engineer. Design the environment and configuration system for the AI Study Buddy backend.

Design: (1) Environment Variables – a complete catalogue (PORT, NODE_ENV, MONGODB_URI, JWT_SECRET and expiries, GEMINI_API_KEY, CHROMA
connection details, upload limits, CORS origins, rate-limit settings) with type, whether required, and a safe default; (2) Config Structure
– a single typed config module that reads and validates env vars at startup and fails fast if any required value is missing; and (3) Secrets
Management – keeping secrets out of source control, a .env.example template, and notes on production secret stores. Present the variable
catalogue as a table plus the config-loading approach. Design only – no code.
```

DELIVERABLES

- ▶ Environment variable catalogue table
- ▶ Typed, validated config module design
- ▶ .env.example template
- ▶ Secrets-management guidance

CHECKLIST

- ☐ All required env vars are catalogued with types
- ☐ Config validates and fails fast at startup
- ☐ A .env.example template is provided
- ☐ Secrets are kept out of source control
- ☐ Production secret handling is addressed

Testing Strategy

OBJECTIVE

Define how the backend is verified at unit, integration, and API levels.

EXPECTED OUTPUT

A testing strategy covering unit, integration, and API testing with tooling and coverage targets.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Senior Backend Engineer in Test. Design the testing strategy for the AI Study Buddy backend.

Design: (1) Unit Testing (recommend a runner such as Jest, what to unit test – services, validators, utilities – and how to mock Gemini, ChromaDB, and the database); (2) Integration Testing (testing controllers with the database and middleware wired together, using an in-memory or disposable Mongo instance and stubbed external services); and (3) API Testing (end-to-end endpoint tests with supertest covering auth, validation, success, and error paths). Recommend folder layout, naming, fixtures, coverage targets, and which external calls to mock vs hit. Provide a representative list of test cases per feature. Design only – no code.
```

DELIVERABLES

- ▶ Layered testing strategy
- ▶ Mocking approach for Gemini/Chroma/Mongo
- ▶ Coverage targets and tooling
- ▶ Representative test-case list per feature

CHECKLIST

- ☐ Unit, integration, and API layers are covered
- ☐ External services have a clear mocking approach
- ☐ Tooling and folder layout are recommended
- ☐ Coverage targets are stated
- ☐ Success and error paths are both tested

Backend Development Roadmap

OBJECTIVE

Sequence the build into phases so the system is delivered incrementally and testably.

EXPECTED OUTPUT

A phased roadmap from authentication through dashboard analytics with goals, dependencies, and exit criteria per phase.

DETAILED PROMPT

COPY & PASTE INTO CLAUDE

```
Act as a Principal Backend Architect and Delivery Lead. Create a phased development roadmap for the AI Study Buddy backend built on Node.js, Express.js, MongoDB Atlas (Mongoose), JWT, bcryptjs, Multer, LangChain, Google Gemini API, ChromaDB, and pdf-parse.

Use these phases in order: Phase 1 Authentication; Phase 2 PDF Upload; Phase 3 ChromaDB Setup; Phase 4 RAG Pipeline; Phase 5 Chat System; Phase 6 Quiz Module; Phase 7 Flashcards; Phase 8 Dashboard Analytics.

For each phase provide: the goal; the specific deliverables and endpoints completed; dependencies on earlier phases; the external setup required (e.g. Mongo Atlas, Gemini key, Chroma instance); the tests that must pass; and clear exit criteria proving the phase is done before the next begins. Present the roadmap as an ordered plan with a short summary table at the end.
```

DELIVERABLES

- ▶ Eight-phase roadmap
- ▶ Deliverables and exit criteria per phase
- ▶ Dependency and external-setup notes
- ▶ Summary roadmap table

CHECKLIST

- ☐ All eight phases are sequenced with goals
- ☐ Each phase lists concrete deliverables
- ☐ Dependencies between phases are explicit
- ☐ Exit criteria gate each phase
- ☐ A summary table is included

Final Backend Code Generation

OBJECTIVE

Combine every prior output into one master prompt that drives complete, file-by-file code generation.

EXPECTED OUTPUT

A single master prompt that, given all earlier deliverables, produces the entire backend codebase one file at a time with nothing skipped.

DETAILED PROMPT

[COPY & PASTE INTO CLAUDE](#)

```
Act as a Principal Backend Architect and Senior Node.js Engineer. Using ALL approved outputs from the previous sections of this handbook – the requirements, architecture, folder structure, schemas, auth design, API specs, PDF and RAG pipeline designs, Gemini and ChromaDB designs, chat, quiz, flashcard, dashboard, middleware, error, validation, security, config, and testing designs – build the complete backend for the AI Study Buddy application.

Requirements: use Node.js, Express.js, MongoDB (Mongoose), JWT, Multer, LangChain, Google Gemini, and ChromaDB. Apply MVC architecture and a Service Layer architecture exactly as defined in the folder-structure section.

Generate the code file by file. For every file: print its full path, then its complete contents, then a one-line note on its responsibility. Follow the approved folder structure precisely and do not skip any file – include package.json, config, models, services, controllers, routes, middleware, utils, the app/server entry, and a .env.example. After the last file, output a tree of everything created and a short run guide (install, env, migrate/seed if any, start). Proceed in dependency order: config and env first, then models, then services, then controllers, then routes, then middleware wiring, then the server entry. If output is long, continue across multiple messages until every file is complete – never summarise a file instead of writing it.
```

DELIVERABLES

- ▶ A single comprehensive master prompt
- ▶ Instruction to emit code file by file in dependency order
- ▶ An explicit 'skip no file' rule
- ▶ A final tree plus run guide instruction

CHECKLIST

- ☐ Prompt references all prior section outputs
- ☐ MVC and service-layer architecture are required
- ☐ Files are generated one by one in dependency order
- ☐ No file may be skipped or summarised
- ☐ A final file tree and run guide are requested